

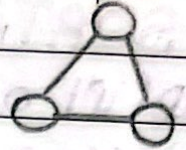
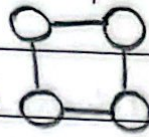
Graphs

Date 20
M T W T F S S

$G(V, E)$

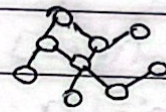
where V = node/vertices

E = Branch/Edge



Used to handle complex scenarios where data has to be organized (recommendation systems, computer networks, social network analysis)

Real-world Example = Google Maps

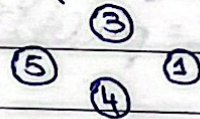
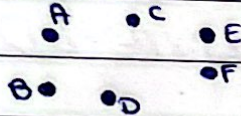


node = city
edges = road

Types Of Graphs

1- Null Graph

A graph is known as a null graph if there are no edges in the graph



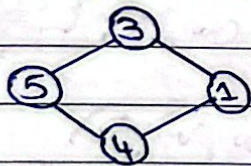
2- Trivial Graph

Graph having only a single vertex, it is also the smallest graph possible



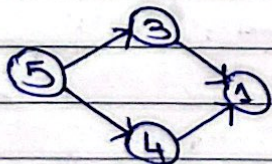
3- Undirected Graph

A graph in which edges have no direction.



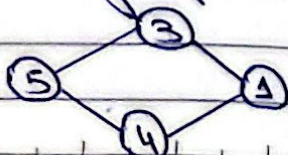
4- Directed Graph

A graph in which edges have a direction



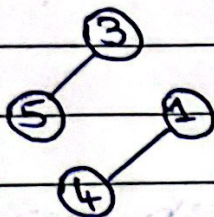
5- Connected Graph

The graph in which from one node we can visit any other node in the graph



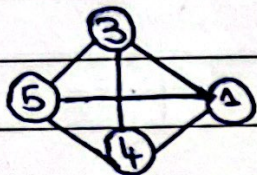
6- Disconnected Graph

The graph in which at least one node isn't reachable from a node



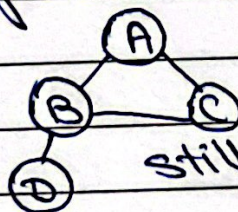
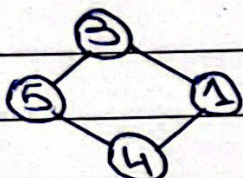
7- Complete Graph

The graph in which from each node there is an edge to each other node (star topology)



8- Cycle Graph

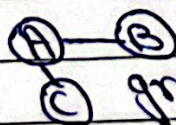
The graph in which the graph is a cycle in itself, the minimum value of degree of each vertex is 2



still cyclic (has one cycle)

9- Acyclic Graph

No cycles



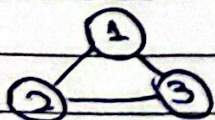
(directed acyclic graph = a

graph that is directed and doesn't have any cycles)

10-

Connected Graph

A graph is connected if every pair of vertices are connected by a path

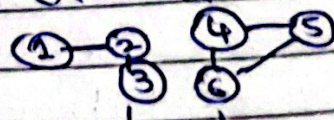
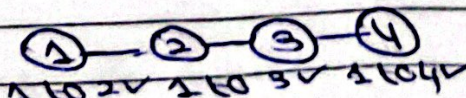


path from 1 to 3

2 to 3

and 1 to 2

So all pairs connected

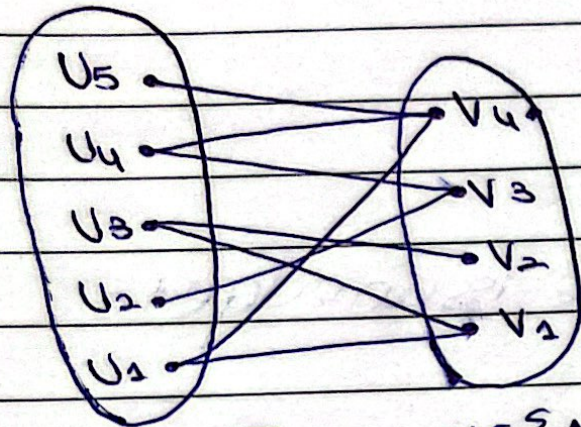


not connected

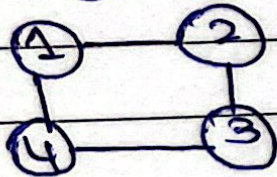
Disconnected graph is disconnected if atleast 1 pair not connected

Bipartite Graph

A graph in which vertex can be divided into two sets such that vertex in each set doesn't contain any edges between them

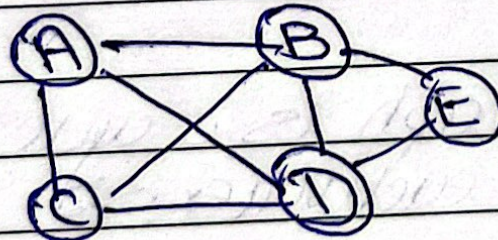
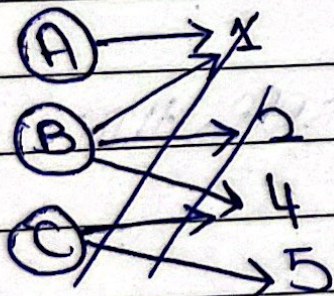


can't connect with their own group's nodes but can ~~connected~~ connect with other group nodes



$$U = \{1, 3\}$$

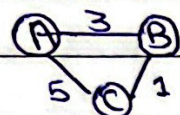
$$V = \{2, 4\}$$



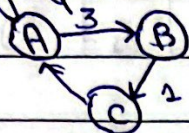
not a star, hence incomplete

13- Weighted Graphs

A graph in which the edges are already specified with suitable weight is known as a weighted graph. It can be further classified as directed weighted graph and undirected weighted graph.



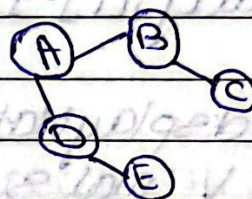
undirected weighted graph



directed weighted graph

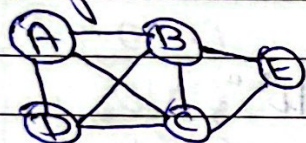
14- Sparse Graphs

A graph with small number of edges



15- Dense Graph

A graph with large number of edges



Representation of Graph

There are multiple ways to store a graph

1- Adjacency Matrix

2- Adjacency Lists

Adjacency Matrix

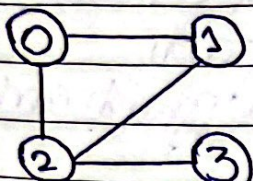
In this method, the graph is stored in the form of the 2D Matrix where rows and columns denote vertices. Each entry represents the weight of those vertices.

→ 2D square Array

→ Fast Lookups

→ Dense Graphs

Example =



	0	1	2	3
0	0	1	1	0
1	1	0	1	0
2	1	1	0	1
3	0	0	1	0

Code Representation =

```
void AddEdge(vector<vector<int>> &mat, int i, int j){
```

```
    mat[i][j] = 1;
```

```
    mat[j][i] = 1;
```

```
}
```

```
void displayMatrix(vector<vector<int>> &mat){
```

```
    int V = mat.size();
```

```
    for(int i=0; i<size V; i++){
```

```
        for(int j=0; j<V; j++){
```

```
            cout<<mat[i][j]<<" ";
```

```
        }
```

Adjacency Lists

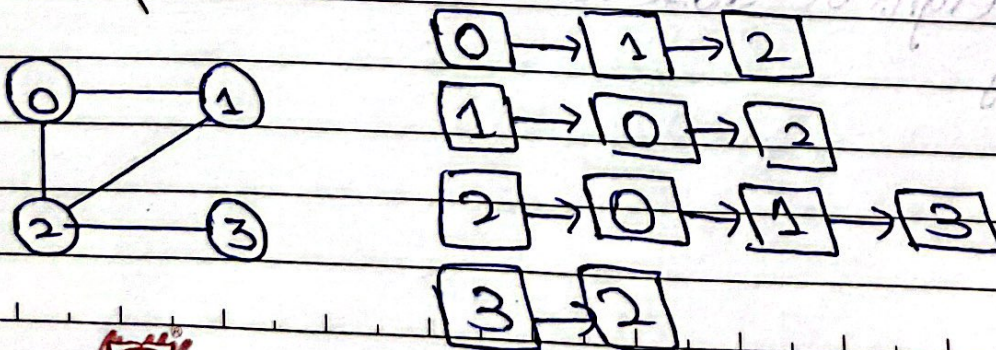
This graph is represented as a collection of linked lists. There is an array of pointer, which points to the edges connected to that vertex.

→ Jagged 2D Array (Array of Arrays)

→ Sparse Graph

→ Fast insertion/deletion of nodes and edges

Example =




```
vector<vector<int>>
vector<int>
list<int>
```

Date 20
M T W T F S S

```
void AddEdge(vector<vector<int>> &adj, int i, int j) {
```

```
    adj[i].push_back(j);
```

```
    adj[j].push_back(i); // undirected
```

```
}
```

```
void displayAdjList(const vector<vector<int>> &adj) {
```

```
    for (int i = 0; i < adj.size(); i++) {
```

```
        cout << i << " : "; // vertex
```

```
        for (int j : adj[i]) {
```

```
            cout << j << " "; // adjacents
```

```
        } cout << endl; }
```

Adding

	Adjacency Matrix	Adjacency List
Adding Edge	$O(1)$	$O(1)$
Removing Edge	$O(1)$	$O(n)$
Initializing	$O(n^2)$	$O(n)$
Checking neighbor	$O(1)$	$O(n)$
Insertion in Node	$O(n)$	$O(1)$
Deletion in Node	$O(n)$	$O(1)$

```
class Graph {
```

```
    int numVert;
```

```
    vector<list<int>> AdjList;
```

```
public:
```

```
    Graph(): numVert(0), undirected(false) {}
```

```
    void AddNode() {
```

```
        AdjList.push_back(list<int>());
```

```
        numVert++;
```

```
    }
```

```
    void AddEdge(int u, int v) {
```

```
        int size = AdjList.size();
```




```
if (u < 0 && v < 0 && u < size && v < size) {
```

```
    AdjList[u].push-back(v);
```

```
    if (undirected) {
```

```
        AdjList[v].push-back(u);
```

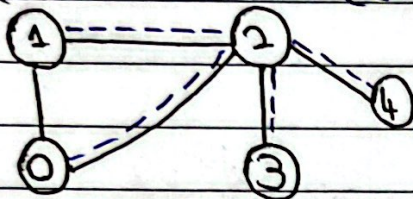
```
    }
```

DFS (Depth First Search) On A Graph

To avoid processing a node multiple times, we use a boolean visited array

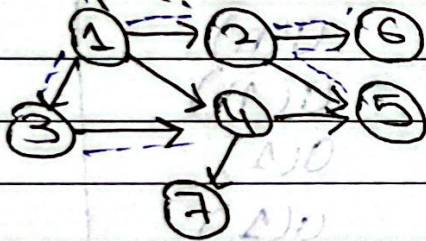
Example

(Undirected Graph)



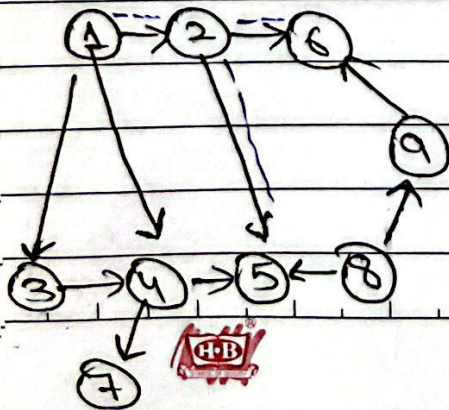
Start at 1, goto 2, then 0, 1 is already visited so backtrack to 2
arr[visited] = 1, 2, 0, 3, 4

Example



Start at 1, backtrack after 6, goto 5, backtrack...
visited = 1, 2, 6, 5, 3, 4, 5, 7

Initially our visit array will be empty but it will fill up with every traversal and if a number has been visited already, it will be skipped



visited = 1, 2, 6, 5, 3, 4, 7, 5

DFS Traversal

DFSHelper(vertex, AdjList, visited):

print vertex

visited[vertex] = true

foreach n in AdjList[vertex]

if visited[n] == false

DFSHelper(n, AdjList, visited)

DFS(AdjList):

vector<bool> visited

Time complexity of $O(V+E)$

Performing DFS and detecting cycle

bool DFS Cycle Und(vertex, parent, AdjList, visited):

visited[vertex] = true

for(^{int} neighbour: adjList[vertex])

if (!visited[neighbour])

if (DFS Cycle Und(neighbour, vertex, AdjList, visited))

return true;

}

} elseif (neighbour != parent)

return true

}

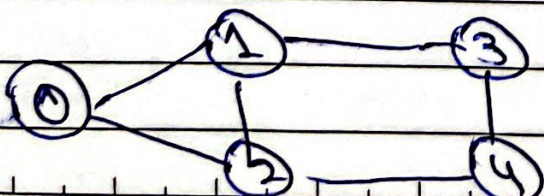
}

BFS (Breadth First Search)

→ Dijkstra's, Kahn's, Prim's

→ Used to find shortest path in unweighted graphs

Example =



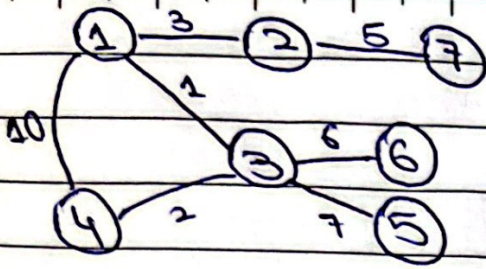
visited = 0, 1, 2, 3, 4

queue = 0, 1, 2, 3, 4

neighbour
of 0

neighbour
of 1

neighbour
of 2



1

BFSHelper(vertex, AdjList, visited)

Queue q

q.push(vertex)

visited[vertex] = true

while q is not empty

temp = q.top()

q.pop()

print temp

for each n in AdjList[temp]

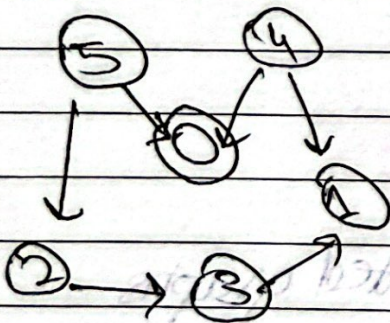
if (visited[n] == false)

visited[n] = true

q.push(n)

Topological Sorting

Topological Sorting for directed acyclic graphs is a linear ordering of vertices such that for every directed edge uv vertex u comes before v in the ordering



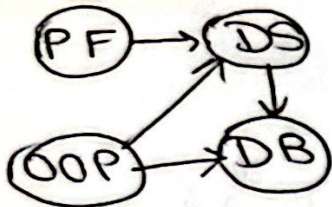
Not possible if graph not DAG

Time complexity $O(V+E)$

Linear Running Time Complexity

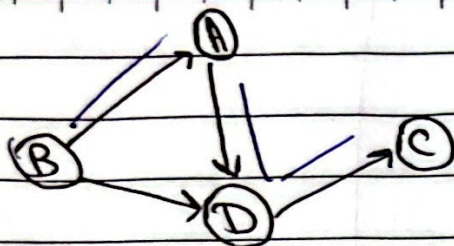
5, 4, 2, 3, 1, 0





PF → DS → DB

Date 20
M T W T F S S



B, A, D, C ✓ valid

A, B, C, D

X no b/c u v condition

bool DFS(vertex, parent, AdjList, visited):

visited[vertex] = true

for each $\frac{1}{n}$ in AdjList[vertex]

if visited[n] == false

if (DFS(n, vertex, AdjList, visited))

return true

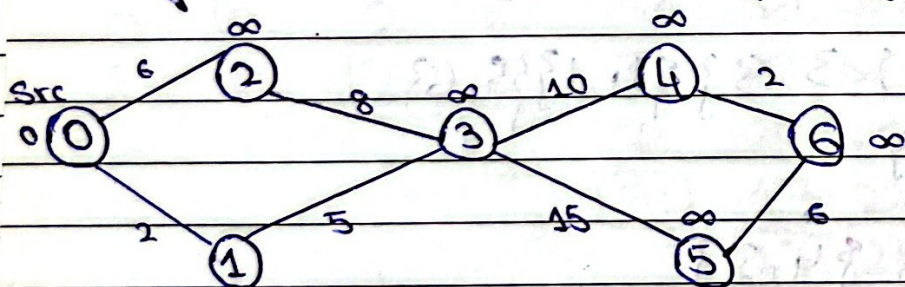
} else if (n != parent) {

return true

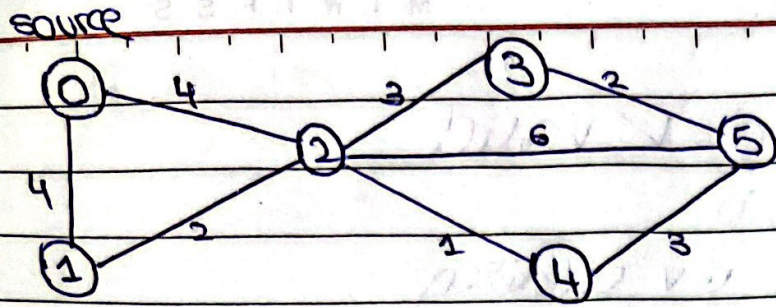
} return false

Dijkstra's Algorithm

Only works with directed acyclic graphs



Source	Init	0	1	2	3	4
0	0	∞	∞	∞	∞	∞
1		2				
2						
3			7			
4				17		
5					22	22
6					19	



{0,0}

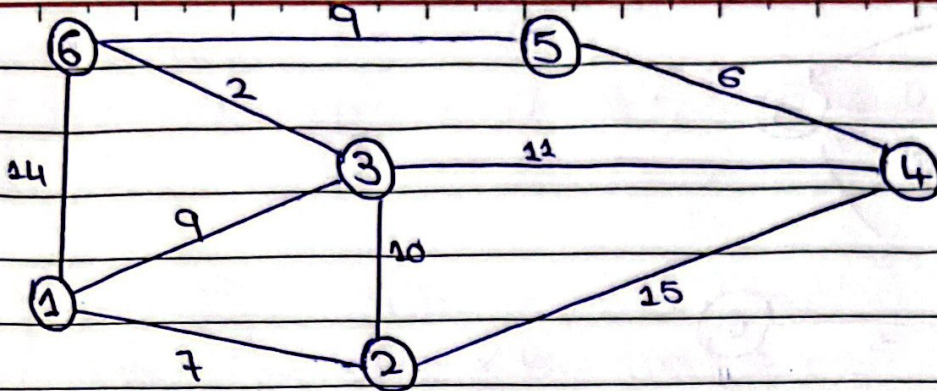
{D,N}

dist, node

	0	1	2	3	4	5
0		4	4			
1	4		6			
2						
3						
4						
5						

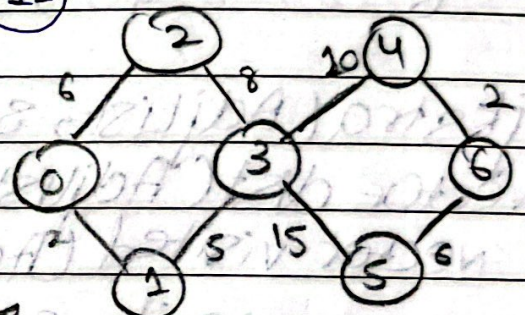
Adjlist

- 0 → {1,4}, {2,4}
- 1 → {0,4}, {2,2}
- 2 → {0,4}, {1,2}, {3,3}, {4,1}, {5,6}
- 3 → {2,3}, {5,2}
- 4 → {2,1}, {5,3}
- 5 → {2,6}, {3,2}, {4,3}



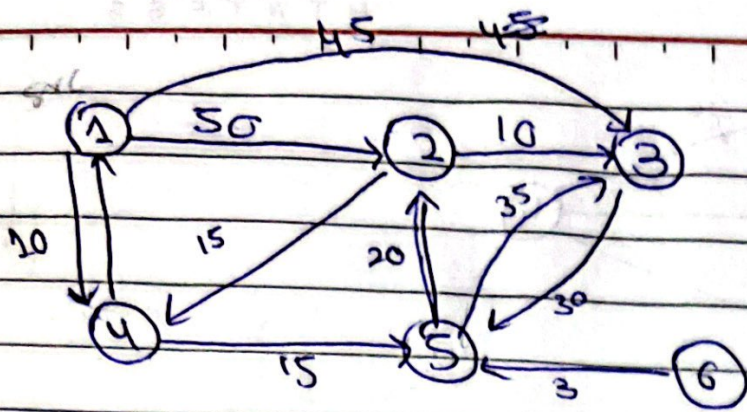
Source	Destination				
	2	3	4	5	6
1	∞	∞	∞	∞	∞
	(7)	9	∞	∞	14
2	(7)	(9)	22	∞	14
3	(7)	(9)	20	∞	(11)
6	(7)	(9)	(20)	20	(11)
4				(20)	
5					

miss wala sawal



	0	1	2	3	4	5	6
0	(0)	2	6	∞	∞	∞	∞
1		(2)	6	7	∞	∞	∞
2			(6)	7	∞	∞	∞
3				(7)	17	22	∞
4					(17)	22	19
						(19)	
6							

25 kg seps
is ke 22 k
kg seps



selected	2	3	4	5	6
	∞	∞	∞	∞	∞
1	50	45	10	∞	∞
4	50	45	10	25	∞
5	45	45	10	25	∞
2	45	45	10	25	∞
3	45	45	10	25	∞
6	45	45	10	25	∞

Dijkstra(AdjList, source):

vector dist(AdjList.size(), INIT_MAX)

vector visited(AdjList.size(), false)

priority-queue pq

dist[src] = 0

pq.push({0, src})

while pq ! empty

{ d, u } = pq.top()

pq.pop()

if (visited[u] == false)

visited[u] = true

for { v, weight } in AdjList[u]

if visited[v] == false and dist[u] + weight < dist[v]

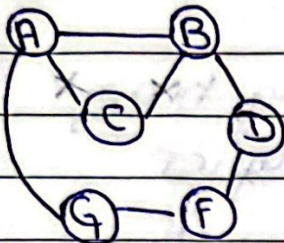
dist[v] = dist[u] + weight

pq.push({ dist[v], v })

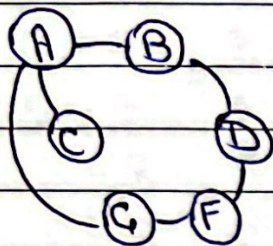
return dist

Spanning Tree

A subset of a graph with V nodes having $V-1$ edges and no cycles

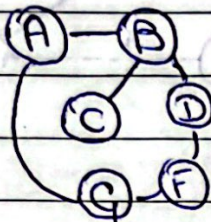


A, B, C is a cycle so we will have to break one of the connections

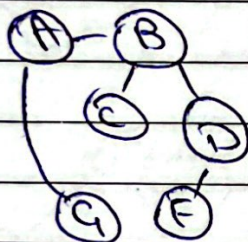
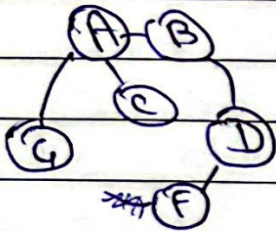


6 nodes

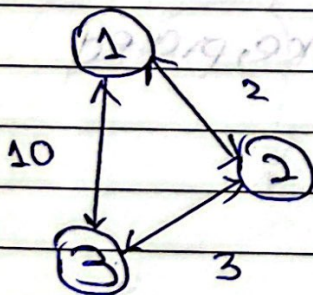
so we need $6 - 1 = 5$ edges



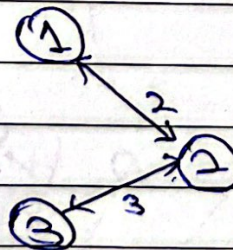
A spanning tree doesn't have any cycles so we will break an additional edge



A minimum spanning tree (MST) is defined as a spanning tree that has minimum weight among all possible spanning trees



⇒
turn to
MST
by
removing
maximum
weight edge



An MST is acyclic

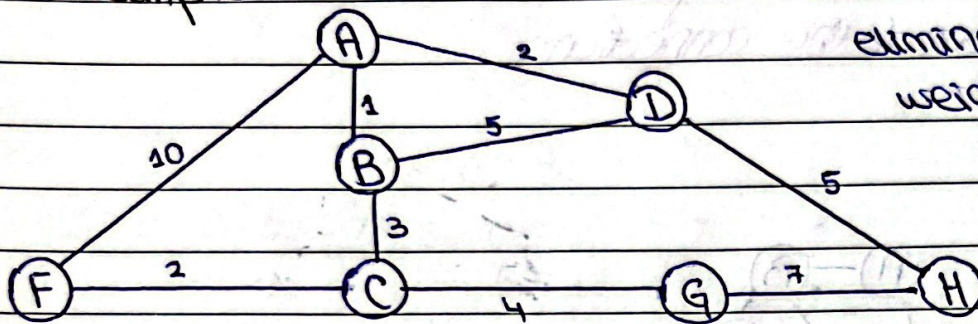
Algorithm to find minimum spanning tree (MST)



Krushkal's Minimum Spanning Tree

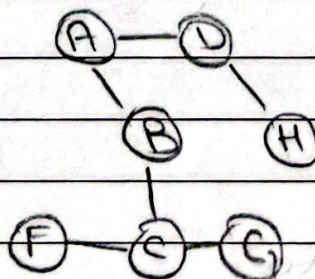
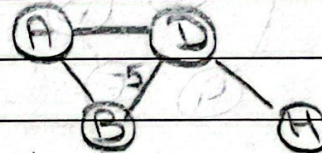
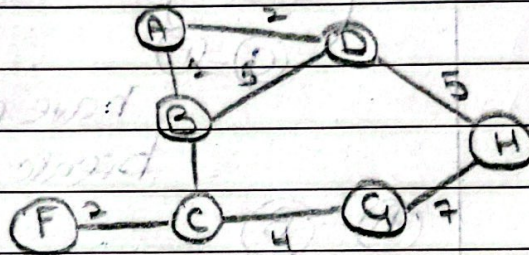
"Krushkal's Minimum Spanning Tree (MST) algorithm is to connect all the vertices of a graph with the minimum total edge weight while avoiding cycles."

Example =



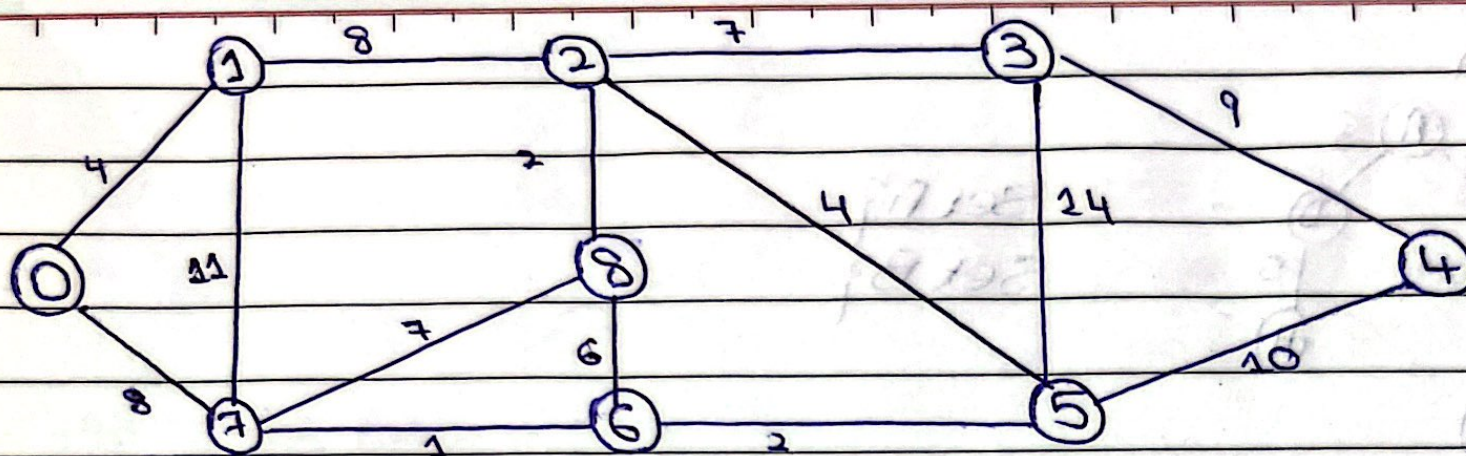
eliminating heavy weight cycles

- A-B 1
- A-D 2
- A-F 10 x
- B-D 5 x
- B-C 3
- C-F 2
- C-G 4
- G-H 7 x
- D-H 5



7 nodes
6 edges

hence, proved



1 - 2 8

1 - 7 11

0 - 1 4

0 - 7 8

6 - 7 1

7 - 8 7

6 - 8 6

6 - 5 2

2 - 8 2

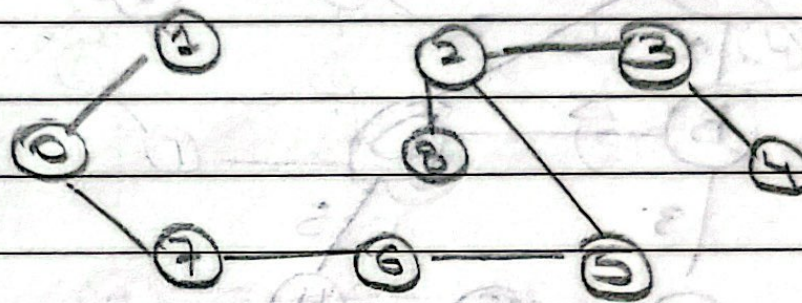
2 - 3 7

2 - 5 4

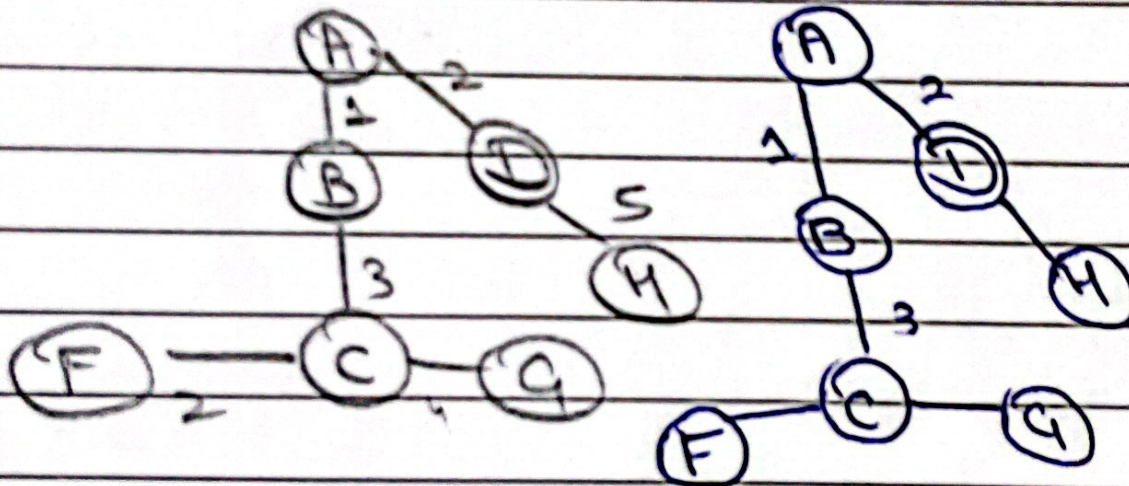
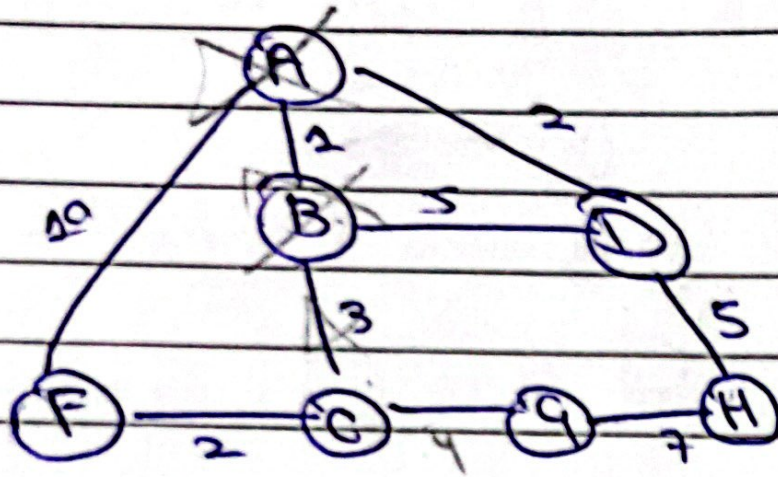
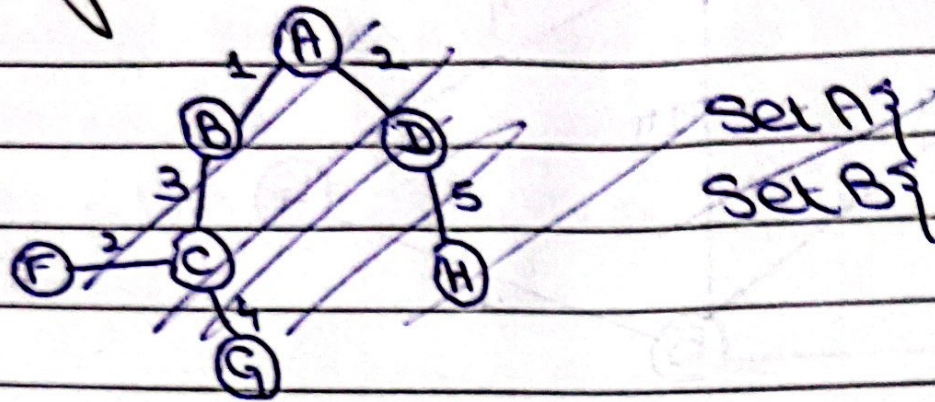
3 - 5 24

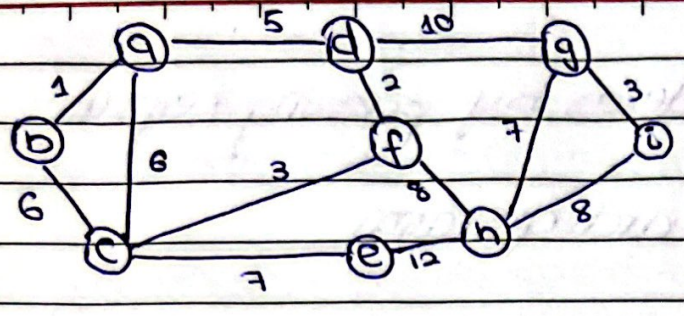
3 - 4 9

4 - 5 10



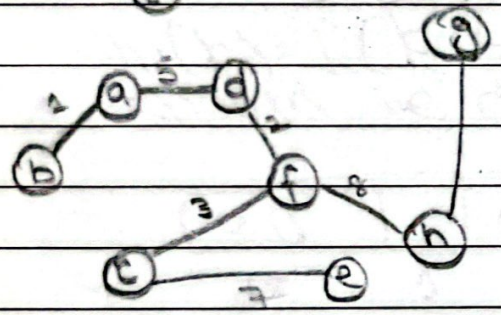
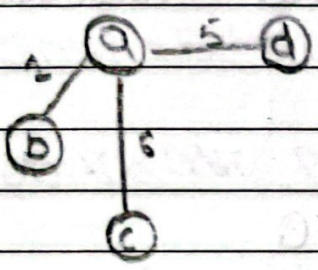
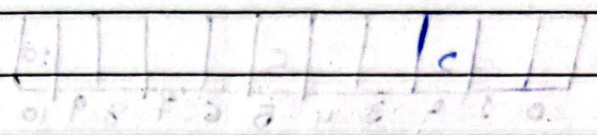
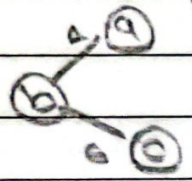
Prim's Algorithm



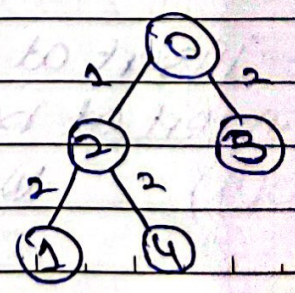
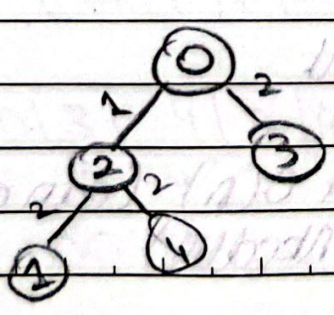


pickly wala ko
bhi dekhna
hoga in prims

starting from b



V	0	1	2	3	4	F ← 11000
0	0	5	2	3	0	
1	5	0	2	3	5	← 0001
2	2	2	0	0	2	
3	2	3	0	0	7	← 0011
4	0	5	2	7	0	



Hashing

Hashing is a method to store and retrieve data from a database in constant time

— It maps keys to indexes by passing keys as input to a hash function

* Hash Table

* Hash Function (K Mod 10, K Mod n, Mid square)

Search key (24, 52, 91, 67, 48, 83)

K Mod 10

$$24 \text{ Mod } 10 = 4$$

want to search for 24, search

(24 / 10)th index. Direct search / Delete.

	0
91	1
52	2
83	3
24	4
	5
	6
67	7
48	8
8	9

Collision

Two distinct values mapping to the same index

Hash Table

Collision Resolution Techniques

Chaining
(Open Hashing)

Open Addressing
(Closed Hashing)

↳ Linear Probing
↳ Quadratic Probing
↳ Double Hashing

24	0
19	1
32 44	2
	3
	4
	5

← collision here

K Mod 6

(24, 19, 32, 44)

0, 1, 2, 2

$$6 \overline{) 24}$$

$$6 \overline{) 19}$$

$$\begin{array}{r} 5 \\ 6 \overline{) 32} \\ \underline{30} \\ 2 \end{array}$$

$$\begin{array}{r} 7 \\ 6 \overline{) 44} \\ \underline{42} \\ 2 \end{array}$$

Hash Table gives index as output when keys are passed through a hashing function

Date 20
M T W T F S S

chaining (open hashing)

Make a linked list and keep mapping/adding elements to it

0	24				
1	19				
2	32	→	44	→	58
3					
4					
5					

Also known as linear probing

Average $O(1)$

worst $O(n)$ (can act as a singly linked list) = disadvantage
~~different deletion~~

~~Advantage of linear probing~~
~~No extra space needed~~

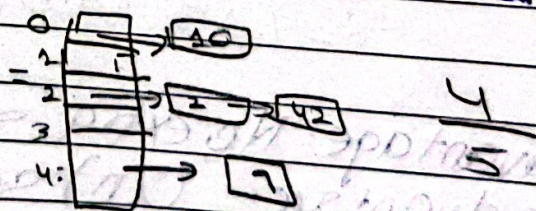
Insertion $O(1)$

Searching $O(n)$ in the worst case

Open Hashing disadvantages

→ Extra space being used for lists while the table is empty

Load Factor = $\alpha = \frac{\text{elements}}{\text{slot}}$



Advantages =

→ Easy Deletion

→ constant time insertion

→ searching can go to $O(n)$ worst case TC

Linear Probing

$$R(k) = k \bmod 10$$

43, 135, 72, 23, 99, 19, 82

$$43 \div 10 = 3$$

$$135 \div 10 = 5$$

$$23 \div 10 = 3 \text{ collision}$$

$$R'(k, i) = (R(k) + i) \bmod 10$$

$$R(k) = 3 \text{ for } 23 \div 10$$

$$R'(k, i) = 3 + 1 = 4 \div 10 = 4$$

$$99 \div 10 = 9$$

$$19 \div 10 = 9$$

20 - next case

82 $\div 10$ next next next

$$(19 \div 10)$$

Advantage = No extra space

Disadvantage = $O(n)$ search time

Primary clustering

secondary clustering

Quadratic Probing

$$h'(k, i) = (h(k) + i^2)$$

19	0
23	1
72	2
43	3
23	4
135	5
82	6
	7
	8
99	9

keys = 42, 16, 91, 33, 18, 27, 36, 62

$$h(k) = k \bmod 10$$

$$h'(k) = (h(k) + i^2) \bmod 10$$

$$42 \div 10 = 2 \quad 16 \div 10 = 6 \quad 91 \div 10 = 1$$

$$33 \div 10 = 3 \quad 18 \div 10 = 8 \quad 27 \div 10 = 7$$

$$36 \div 10 = 6 \text{ already filled}$$

$$h'(k) = (h(k) + i^2) \bmod 10$$

$$h'(k) = (6 + 1^2) \div 10$$

$$h'(k) = 7 \div 10$$

7 also filled

$$h''(k) = ((h(k) + i^2) \bmod 10$$

$$6 + 2^2 \bmod 10$$

$$6 + 4$$

$$10 \div 10 = 0$$

$$62 \div 10 = 2 \Rightarrow 2 + 1 \div 10 \Rightarrow 2 + 4 \div 10 \Rightarrow 6 \div 10$$

$$2 + 9 \div 10 = \text{Kartay raho}$$

Advantage No extra space
 Primary clustering resolved

Disadvantage
 Search $O(n)$

Secondary clustering

No guarantee of finding slot

Double Hashing

20, 34, 45, 70, 56

$$h_1(k) = k \text{ Mod } 11$$

$$h_2(k) = 8 - (k \text{ Mod } 8)$$

$$(h_1(k) + i h_2(k)) \text{ Mod } 11$$

$$20 \text{ Mod } 11 = 9 \quad 34 \text{ Mod } 11 = 1$$

$$45 \text{ Mod } 11 = 1$$

$$h_1(k) + i h_2(k) \text{ Mod } 11$$

$$1 + 1(3) \text{ Mod } 11$$

$$4 \text{ Mod } 11$$

$$= 4$$

$$h_2(k) = 8 - (45 \text{ Mod } 8)$$

$$8 - 5$$

$$h_2(k) = 3$$

70

$$70 \text{ Mod } 11 = 4$$

$$h_2(k) = 8 - (70 \text{ Mod } 8) = 8 - 6 = 2$$

$$4 + 2 \text{ Mod } 11$$

$$6 \text{ Mod } 11 = 6$$

$$56 \text{ Mod } 11 = 1$$

$$8 - (56 \text{ Mod } 8)$$

$$1 + 8 = 9 \text{ also occupied}$$

$$1 + 2(8) \cdot 11$$

$$16 + 1 = 17 \cdot 11 = 6 \text{ bhari v hai}$$

$$1 + 3(8) = 25 \cdot 11 = 3$$

Advantage = No extra space

No primary or secondary clustering

Disadvantage $O(n)$ search time

